

IMAGE PROCESSING PLATFORM

A scalable, asynchronous image processing platform built on microservices and Kubernetes



Microservices
Go + Python



Async Queue
RabbitMQ + Outbox



Kubernetes
HPA autoscaling

Engineering Thesis · 2026

AGENDA

› 01 Problem & motivation
› 02 System overview & architecture
› 03 Microservices deep dive (×3)
› 04 Technology stack
› 05 Technology explainers: MinIO · RabbitMQ · Outbox · SPA · HPA
› 06 End-to-end data flow
› 07 Data model
› 08 Key architectural decisions
› 09 Kubernetes & HPA
› 10 Project status & roadmap

Thesis goal: Design and implement a production-grade, horizontally scalable image processing platform that demonstrates distributed systems patterns — async messaging, transactional outbox, and cloud-native orchestration.

> The platform is built incrementally across 5 chapters, each introducing a new architectural layer — from a bare REST API through workers, frontend, and Kubernetes.

PROBLEM & MOTIVATION

THE CHALLENGE

› Modern SaaS applications process thousands of images per minute — resize, watermark, format conversion

› Synchronous processing blocks the HTTP connection for seconds, degrading user experience

› A monolithic architecture cannot scale image processing independently from the API

› A single server crash can silently lose queued work with no recovery mechanism

REQUIREMENTS

› REST API for file upload, metadata retrieval, and result download

› Asynchronous processing — client gets an immediate `202 Accepted`

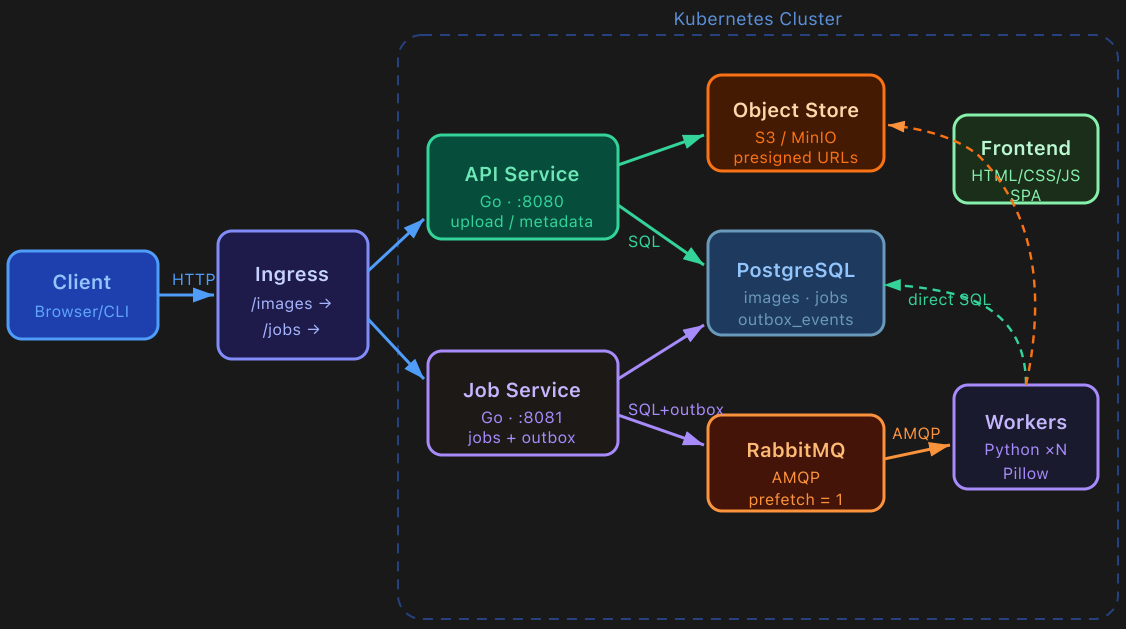
› Reliability — no job can be lost even if a service crashes mid-flight

› Horizontal scaling — add more workers under load, remove them when idle

› Security — reject malicious uploads before they reach the processing stage

The core problem: how do you decouple the "accept work" path from the "do work" path, guarantee delivery, and still scale each independently — without over-engineering a small team project?

ARCHITECTURE OVERVIEW



API Service (Go)

Accepts uploads, validates files, store originals in S3, record metadata PostgreSQ Exposes presigned download URLs.

Job Service (Go)

Creates processing jobs and reliably enqueues them to RabbitMQ

using the *transactio outbox* pattern.

Workers (Python)

Consume one message a a time, process th image with Pillow, upload the result to S and updat the job status directly in PostgreSQ

Each service is independently deploy and scales separatel the core promise of microservices.

MICROSERVICE: API SERVICE

RESPONSIBILITY

The API Service is the single entry point for file ingestion. It accepts raw image uploads over HTTP, validates them, persists the file to object storage, and records metadata in PostgreSQL. It also serves presigned download URLs for both originals and processed results.

ENDPOINTS

`POST /api/v1/images`	Upload a new image
`GET /api/v1/images`	List all images
`GET /api/v1/images/:id`	Image metadata + download URL

Language: Go 1.22 · **Port:** :8080 · **Module:** `image-platform/api-service`

UPLOAD PIPELINE (PER REQUEST)

› Parse `multipart/form-data` — stream file to memory

› Read image header with `image.DecodeConfig` — no pixel decode

› Reject if file size > `MAX_UPLOAD_SIZE_MB` or pixels > `MAX_MEGAPIXELS`

› Generate UUID object key, stream file to S3

› Insert row into `images` table in PostgreSQL

› Return `201 Created` with image metadata JSON

EXTERNAL DEPENDENCIES

PostgreSQL `images` table — metadata

S3 / MinIObinary file storage

> The API Service is stateless — it holds no in-memory state between requests. Any replica can handle any request, which makes horizontal scaling trivial.

MICROSERVICE: JOB SERVICE

RESPONSIBILITY

The Job Service manages the lifecycle of processing jobs. It receives a request to process an image, creates a job record in PostgreSQL, and *reliably* enqueues the job to RabbitMQ using the transactional outbox pattern. It also exposes an endpoint to query job status.

ENDPOINTS

`POST /api/v1/jobs`	Create a processing job → 202
`GET /api/v1/jobs/:id`	Get job status + result URL
`GET /api/v1/jobs`	List jobs (filterable by status)

Language: Go 1.22 · **Port:** :8081 · **Module:** `image-platform/job-service`

JOB CREATION — TRANSACTIONAL OUTBOX

- Begin PostgreSQL transaction
- Insert row into `jobs` table (status = PENDING)
- Insert row into `outbox_events` table (published = false)
- Commit — both rows saved atomically
- Return `202 Accepted` immediately
- Background goroutine polls outbox → publishes to RabbitMQ

EXTERNAL DEPENDENCIES

PostgreSQL `jobs` + `outbox_events`

RabbitMQpublish via outbox poller

> The outbox goroutine runs every 500 ms (configurable via `OUTBOX_POLL_MS`). It batches unpublished events, publishes each to RabbitMQ, and marks them `published = true`.

MICROSERVICE: PROCESSING WORKER

RESPONSIBILITY

The Processing Worker is a stateless consumer that picks up one job at a time from the RabbitMQ queue, downloads the source image from S3, applies the requested transformation using Pillow, uploads the result back to S3, and updates the job record in PostgreSQL — without any HTTP calls to other services.

SUPPORTED OPERATIONS

resize	Scale to given width × height
convert	Change format (JPEG → PNG etc.)
watermark	Overlay text or image
thumbnail	Fast square crop + resize

Language: Python 3.12 · **Library:** Pillow · **Queue client:** pika (AMQP)

PROCESSING LOOP (PER MESSAGE)

› Consume 1 message from RabbitMQ (`prefetch_count=1`)

› Parse job payload — get `job_id`, `operation`, `parameters`

› Set job status → PROCESSING in PostgreSQL

› Download source image from S3

› Apply Pillow transformation (resize / convert / watermark)

› Upload result to S3 with a new object key

› Set job status → COMPLETED + store result key in PostgreSQL

› Acknowledge message to RabbitMQ

EXTERNAL DEPENDENCIES

RabbitMQ	consume jobs (AMQP)
S3 / MinIO	download source, upload result
PostgreSQL	update job status directly

> If processing fails, the worker sets status → FAILED with an error message and **nacks** the RabbitMQ message (optionally requeuing it). The `attempt_count` field tracks retries.

TECHNOLOGY STACK

SERVICES

API Service	Go 1.22	net/http, minio-go
Job Service	Go 1.22	amqp091-go
Worker	Python 3.12	Pillow, pika
Frontend	HTML/CSS/JS	SPA, fetch API

INFRASTRUCTURE

Database	PostgreSQL 16	JSONB, UUID
Object Storage	S3 / MinIO	presigned URLs
Message Queue	RabbitMQ 3.13	AMQP 0-9-1
Orchestration	Kubernetes	HPA, PVC, Secrets
Local dev	Docker Compose	

Why Go for services? Go compiles to a single binary, has excellent concurrency primitives (goroutines) for the outbox poller, and produces small Docker images. Both services start in under 100ms.

Why Python for workers? The Python imaging ecosystem (Pillow) is mature and well-documented. Workers are stateless and CPU-bound — easy to scale horizontally regardless of language runtime.

TECHNOLOGY: MINIO — OBJECT STORAGE

WHAT IS MINIO?

MinIO is an open-source, S3-compatible object storage server. It stores files as *objects* (blobs) identified by a *bucket* and an *object key* (a string path). Unlike a relational database, it has no schema — it stores any file type at any size.

In this project, MinIO runs as a Docker container in local development. In production on Kubernetes it is replaced by any S3-compatible cloud service (AWS S3, Google Cloud Storage, etc.) — **the Go code does not change** because the API is identical.

WHY NOT STORE FILES IN POSTGRESQL?

› Binary files in a database bloat WAL logs and make backups slow

› S3 scales independently from the database — no IOPS contention

› Object storage is designed for large, immutable blobs — it's much cheaper per GB

› Presigned URLs let clients download directly from storage, bypassing the API server entirely

HOW IT IS USED HERE

API Service	Uploads originals on POST /images
API Service	Generates presigned GET URL (TTL 60 min)
Worker	Downloads source, uploads result
Client	Downloads result directly via presigned URL

Presigned URL: a time-limited, cryptographically signed URL that grants read access to a single object without any credentials. The client downloads the file directly from S3 — the API server is not in the data path.

> The Go client library is `minio-go`. Because MinIO speaks the same S3 API, switching to AWS S3 in production is a one-line config change — no code rewrite.

TECHNOLOGY: RABBITMQ — MESSAGE QUEUE

WHAT IS RABBITMQ?

RabbitMQ is a message broker — a server that accepts messages from producers and delivers them to consumers via named queues. It implements the **AMQP 0-9-1** protocol.

A producer (Job Service) publishes a message to an *exchange*. The exchange routes it to a *queue*. A consumer (Worker) pulls from that queue, processes the message, and sends an *acknowledgement* (ack) when done. Only then does RabbitMQ delete the message.

WHY RABBITMQ INSTEAD OF DIRECT HTTP?

› **Decoupling:** Job Service does not need to know how many workers exist or where they are

› **Reliability:** messages persist in the queue even if all workers are down

› **Load leveling:** workers consume at their own pace — no API timeout, no back-pressure on the HTTP layer

› **Scaling:** add workers without changing any configuration in Job Service

PREFETCH_COUNT = 1

This setting tells RabbitMQ to send at most **1 unacknowledged message** to each worker at a time. A new message is only dispatched after the previous one is acked.

› Without it: RabbitMQ would push all queued messages to the first available worker, starving others

› With it: a slow worker (large image) does not block faster ones — work distributes fairly

What happens if a worker crashes mid-job? Because the message was not acked, RabbitMQ automatically requeues it and delivers it to another worker — no job is lost.

RabbitMQ also has a **Management UI** (port 15672) that shows live queue depth, message rates, and consumer counts — useful for monitoring during load tests.

PATTERN: TRANSACTIONAL OUTBOX

THE PROBLEM IT SOLVES

A naive implementation of "create job + publish to queue" would do two separate operations:

1. INSERT job into PostgreSQL
2. Publish message to RabbitMQ

If the service crashes **between steps 1 and 2**, the job exists in the database but RabbitMQ never received the message — the job is permanently stuck with no worker ever picking it up.

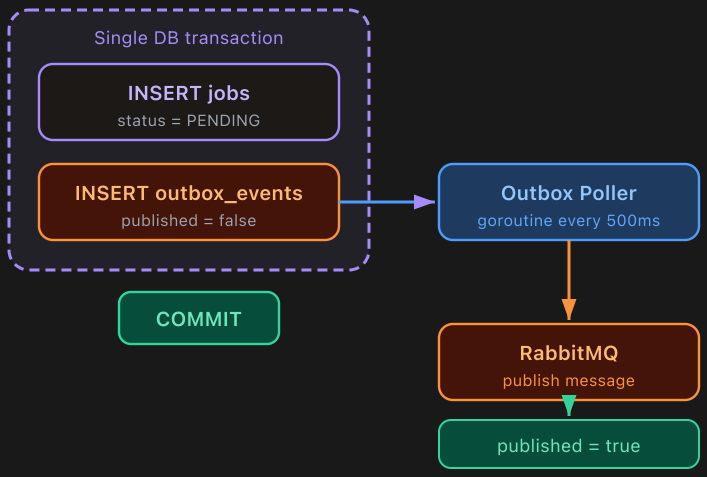
THE SOLUTION

Write the message to a database table (`outbox_events`) **in the same transaction** as the job. A separate background goroutine (the *poller*) reads unpublished rows and forwards them to RabbitMQ.

› If the service crashes before publishing — the row is still in the DB, poller will retry

› If the service crashes after publishing but before marking as published — the message is re-sent (at-least-

once), worker must be idempotent



Key insight: by using the database as a reliable intermediate store, we reduce a distributed two-phase commit problem to a simple local transaction. The poller handles the async delivery with automatic retry — no external coordinator needed.

TECHNOLOGY: SPA — SINGLE PAGE APPLICATION

WHAT IS AN SPA?

A Single Page Application is a web frontend that loads **once** (a single HTML file) and then dynamically updates the content using JavaScript — without full page reloads. Navigation between "pages" is handled client-side by manipulating the DOM.

In this project, the frontend is a plain HTML/CSS/JavaScript SPA — no framework required. It communicates with the backend exclusively through REST API calls using the browser's native `fetch()` API.

WHY SPA OVER SERVER-RENDERED HTML?

› The backend is already an API — no templating layer needed

› A single static HTML file can be served from any CDN or Nginx container

› Faster perceived performance — only data (JSON) is fetched after first load, not full HTML pages

› Clean separation: backend owns data, frontend owns presentation

FRONTEND RESPONSIBILITIES

› Upload form — drag and drop image, send `multipart/form-data`

› Display upload progress and returned image metadata

› Job creation form — select operation and parameters

› Poll `GET /api/v1/jobs/:id` every few seconds to show live status

› Display the processed result image when the job completes

SINGLE BASE URL VIA INGRESS

Because Kubernetes Ingress routes both `/api/v1/images` and `/api/v1/jobs` from a single hostname, the frontend only needs **one configured base URL** — it never knows or cares that two separate services exist behind it.

> Chapter 4 of the thesis. Currently the backend is fully functional — the SPA is the remaining user-facing layer.

TECHNOLOGY: HPA — HORIZONTAL POD AUTOSCALER

WHAT IS HPA?

Horizontal Pod Autoscaler is a Kubernetes controller that automatically adjusts the number of running Pods in a Deployment based on observed metrics — typically CPU or memory utilisation.

When CPU usage exceeds the configured threshold, Kubernetes adds more replicas. When usage drops, it removes them. The process is fully automatic — no human intervention needed.

HORIZONTAL VS VERTICAL SCALING

› **Horizontal (HPA):** add more identical pods — load is distributed. Preferred for stateless services.

› **Vertical:** give each pod more CPU/RAM. Has limits, requires restart, harder to automate.

Workers are stateless — each one processes one job independently. Adding 10 workers = 10× throughput. This is the ideal case for horizontal scaling.

HPA CONFIGURATION IN THIS PROJECT

api-servicemin 1 · max 5 · CPU target 70%
job-servicemin 1 · max 3 · CPU target 70%
workersmin 1 · max 10 · CPU target 80%

Why 1–10 workers? Image processing (decode → transform → encode) is CPU-intensive. Under a burst of upload requests, worker CPU saturates quickly. A wide autoscaling range lets the platform absorb traffic spikes without pre-provisioning expensive hardware at all times.

> Chapter 5 of the thesis includes load tests that demonstrate HPA kicking in — measuring throughput with 1 worker vs 10 workers under identical request rates.

END-TO-END DATA FLOW



Steps 1–2: Synchronous path

The API Service validates the file header (without fully decoding it), rejects pixel bombs, and persists the original to S3 and metadata to PostgreSQL — all within a single HTTP request cycle.

Step 3: Hand-off

The Job Service creates a job record and an outbox event in a single database transaction. It immediately returns `202 Accepted` with the job ID — the client is never blocked waiting for processing to complete.

Steps 4–5: Async path

A background goroutine polls unpublished outbox events and pushes them to RabbitMQ. Workers consume one message at a time, process the image, and write the result directly to S3 and PostgreSQL.

DATA MODEL

images

```
id UUID PK
original_filename TEXT
object_key TEXT UNIQUE
content_type TEXT
file_size BIGINT
width, height INT
created_at TIMESTAMPTZ
```

jobs

```
id UUID PK
image_id UUID FK → images
operation TEXT
parameters JSONB
status ENUM
result_object_key TEXT
created / started / finished_at
```

outbox_events

```
id UUID PK
job_id UUID FK → jobs
payload JSONB
published BOOLEAN DEFAULT false
created_at TIMESTAMPTZ
```

Partial index WHERE published=false

Job status lifecycle: PENDING → PROCESSING → COMPLETED / FAILED

The `attempt_count` field and `error_message` column allow workers to record retries and failure reasons without extra tables.

JSONB parameters: Using JSONB for job parameters means the schema does not need to change when new operation types are added — a resize job carries `{"width":800,"height":600}`, a watermark job carries different keys. No migrations needed for new operations.

KEY ARCHITECTURAL DECISIONS

TRANSACTIONAL OUTBOX

The job row and the outbox_event row are inserted in a single database transaction. A separate goroutine polls unpublished events and forwards them to RabbitMQ.

Guarantees at-least-once delivery even if the service crashes after writing to the DB but before publishing to the queue.

WORKER → DB DIRECT

Workers update job status directly in PostgreSQL, without any HTTP call back to Job Service. This removes a synchronous dependency between services.

Job Service does not become a bottleneck or single point of failure for the processing pipeline.

PREFETCH_COUNT = 1

Each worker fetches exactly one message from the queue at a time. RabbitMQ only dispatches the next message after acknowledgement.

Ensures fair load distribution across replicas — a slow worker does not starve faster ones.

PIXEL BOMB PROTECTION

`image.DecodeConfig` reads only the file header, never decoding pixels into memory. The megapixel check rejects the file before it is stored anywhere.

Prevents memory-exhaustion DoS attacks via crafted images (e.g. a 1 × 100 000 000 px PNG).

PRESIGNED URL TTL

Processed results are served directly from S3 via a time-limited presigned URL (default 60 min). The API never streams binary file data to the client.

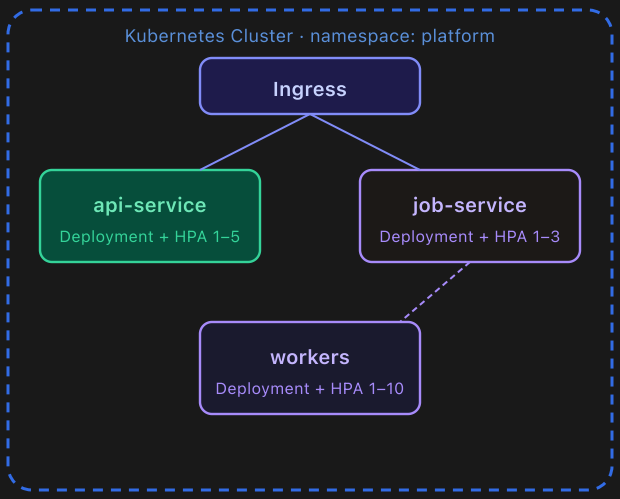
Eliminates a bandwidth bottleneck at the API layer and offloads large file transfers entirely to the object store.

SINGLE INGRESS

A Kubernetes Ingress routes `/images` to API Service and `/jobs` to Job Service. The client uses one base URL — services scale independently behind it.

Decouples client configuration from the internal service topology entirely.

KUBERNETES DEPLOYMENT & SCALING



HORIZONTAL POD AUTOSCALER

api-service	1 – 5 replicas · CPU 70%
job-service	1 – 3 replicas · CPU 70%
workers	1 – 10 replicas · CPU 80%

STORAGE & CONFIG

- PostgreSQL & MinIO — PersistentVolumeClaims
- DB credentials — Kubernetes Secrets
- Service config — ConfigMaps
- Ingress routing — path-based rules

> Workers have the widest scaling range (1–10) because image processing is the most CPU-intensive part of the pipeline. Under heavy load, Kubernetes automatically adds pods; when the queue drains, it scales back down.

PROJECT STATUS & ROADMAP

Chapter 1 API Service · PostgreSQL · MinIO
✓ DONE

Chapter 2 Job Service · RabbitMQ · Outbox
✓ DONE

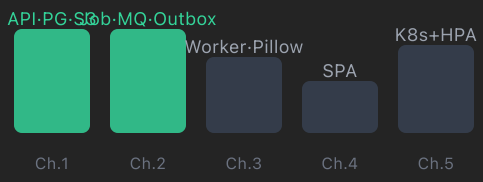
Chapter 3 Python Worker · Pillow processing
TODO

Chapter 4 Frontend SPA · HTML/CSS/JS
TODO

Chapter 5 Kubernetes · HPA · load testing
TODO

Overall progress 2 / 5 chapters complete

New components introduced per chapter



What's already working: full upload pipeline — REST API, file validation, S3 storage, PostgreSQL metadata, job creation, transactional outbox, and RabbitMQ publishing. End-to-end smoke tests pass in Docker Compose.

Next step — Chapter 3: implement the Python worker that consumes jobs from RabbitMQ, applies Pillow transformations (resize, convert, watermark), and writes results back to S3 and PostgreSQL.

SUMMARY

WHAT WAS BUILT

› Two production-grade Go microservices with full REST APIs

› Reliable async messaging via the transactional outbox pattern

› S3-based object storage with presigned URL delivery

› PostgreSQL schema with outbox, partial indexes, and auto-trigger

› Docker Compose local environment with health checks

› Kubernetes manifests with HPA for all services

PATTERNS DEMONSTRATED

› Transactional Outbox — reliable event delivery

› At-least-once delivery with idempotent workers

› Loose coupling — workers never call services over HTTP

› Security-first file ingestion — pixel bomb protection

The platform demonstrates that a small team can build a reliable, scalable distributed system by applying well-

known patterns (outbox, prefetch, presigned URLs) rather than complex custom infrastructure.

2 services done	3 to go
6 arch. decisions	∞ horizontal scale



QUESTIONS?

Image Processing Platform · Engineering Thesis · 2026

Stack
Go · Python · PostgreSQL
RabbitMQ · S3/MinIO · Kubernetes

Key patterns
Transactional Outbox
Async queue · Presigned URLs · HPA

Speaker notes